Inside the Chipless Reader

Where the pattern goes, how it gets there, and what every part of the circuit does along the way, followed from the Mac to the tag and back.

Firmware 0.3 · STM32F407G-DISC1 · RF front-end not yet built · November 2, 2026

The short answer: nothing sends the pattern over the radio.
The tag has no chip, no battery and no message. Its pattern is built into its copper as a row of resonators. The reader transmits a plain tone, one frequency at a time, and the tag absorbs energy only where it has a resonator. The pattern first appears as *dips in the returned signal*, and the STM32 works it out by measuring those dips.

The pattern is only ever *sent as data* in two places, both over USB: from the Mac to the STM32 when you add a tag to the database, and from the STM32 to the Mac when it reports a read.

No real RF part or tag has been tested yet. The USB link, decoding and flash storage have been run on the real STM32, but the radio and tag are simulated. Every signal value here shows what the design is *expected* to do. Treat it as a guide for bring-up, not as a measurement.

Where each number comes from

Recorded	Captured for this document by running your unmodified `firmware/core` code with a recording layer: every SPI word, PA switch, delay and USB byte. Noise was on, as in the default settings.
Datasheet	The chain values in `app_config.h` and `docs/DESIGN_NOTES.md`.
Estimate	The simulator's channel numbers: −70 dB leakage, −52 dB echo at 20 cm, −20 dB notch, Q 2000. They are not measured. Other calculations made for this document are labelled as estimates where they appear.

Contents

1. Where the pattern goes 2
2. The whole circuit 3
3. One read, event by event 5
4. Following a 1 and a 0 through the circuit 6
5. Every part, in signal order 8
6. Turning the voltages into the pattern 17
7. The pattern as bytes on USB 19
8. Limits, risks and faults 21
9. What is real, sources and terms 23

1. Where the pattern goes

The pattern `1010110011` exists in four different forms during its life. Only one of them is a message being sent from one place to another in the ordinary sense, and that one is on USB.

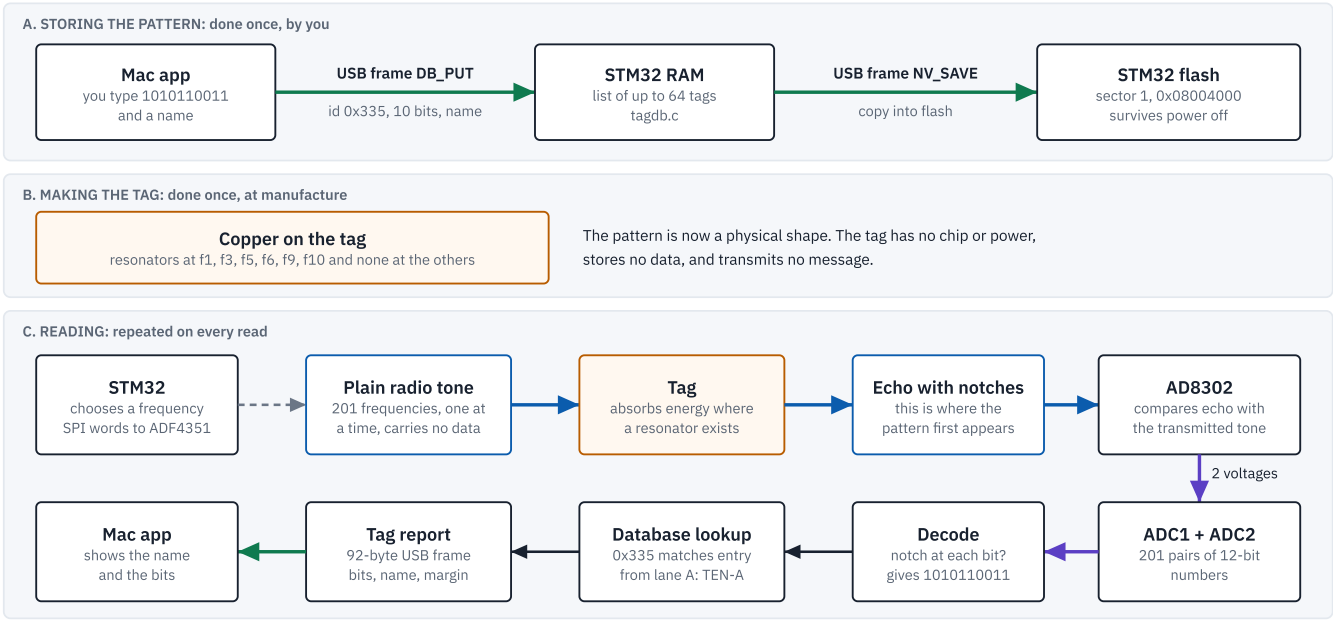


Figure 1. The three separate activities involving the pattern. Green arrows carry the pattern as digital data. The blue radio path carries no data at all.

FORM	WHERE IT IS	IS IT BEING SENT?
Text and number	Mac app, then STM32 RAM and flash	**Yes, over USB.** `1010110011` becomes the number `0x335` in a `DB_PUT` frame. This tells the reader what name belongs to that pattern. It does not touch the tag.
Copper shape	On the tag	**No.** A resonator is present for each 1 and absent for each 0. It is fixed at manufacture and can't be rewritten.
Notches in a signal	In the echo, from the tag to the RX antenna and through to the AD8302	**Not as a message.** The tag is passive. It changes how much of the reader's own tone comes back at each frequency.
Decoded number	STM32 RAM, then the USB reply	**Yes, over USB.** After the decode, `0x335` and the name `TEN-A` go to the Mac in a tag report.

What the pattern means physically

The 860–865 MHz band is divided into ten cells, each 500 kHz wide, with one possible resonator at each centre. Patterns are written with the lowest frequency first.

f1	f2	f3	f4	f5	f6	f7	f8	f9	f10
1	0	1	0	1	1	0	0	1	1

Inside the software, f1 is bit 0. So the ID is 1 + 4 + 16 + 32 + 256 + 512 = 821, which is `0x335`.

Proof that the radio side carries no pattern

The control signals were recorded during one read with no tag, then with three different tags (`TEN-A`, `TEN-B` and `TEN-C`). All four runs produced **exactly the same 620 control events** in the same order. The reader does the same thing whichever tag is present and only the measured results differ.

The bit frequencies aren't used to choose what is transmitted, either. The sweep covers the whole band in 25 kHz steps. The bit frequencies are only used afterwards, to decide where to look in the results.

2. The whole circuit

The easiest way to understand this circuit is as a **small home-built network analyser**. It generates a known tone, keeps a copy as a reference, sends the tone through an unknown path that includes the tag, and compares what returns with the reference. That comparison, repeated at 201 frequencies, is the entire measurement.

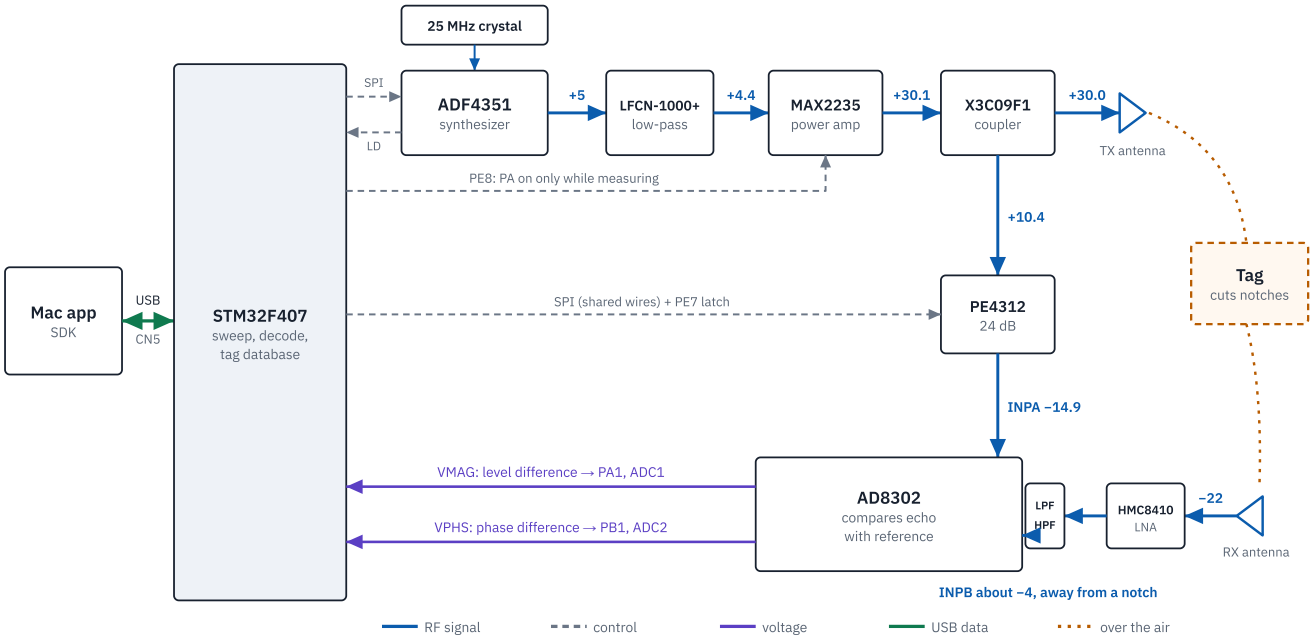


Figure 2. The complete circuit at the default settings. Levels are in dBm, from datasheet values and the estimated echo at 20 cm.

How each part is used

PART	ITS JOB	ACTIVE WHEN	STM32 CONTROL	ROLE IN THE PATTERN
Mac app + SDK	User interface, frames	Always	USB CDC, CN5	Stores patterns; shows results
STM32F407	Runs everything, decodes	Always	·	Decides each bit and looks up the name
25 MHz reference	Frequency standard	Always	None	Sets the accuracy of every frequency
ADF4351	Generates the tone	Always; retuned in sweeps	SPI2, LE PB12, CE PE9, LD PE10	Visits each frequency in turn
LFCN-1000+ (TX)	Removes harmonics	Passive	None	Keeps the tone clean
MAX2235	Amplifies to about 1 W	During measurements	SHDN from PE8	Makes the echo strong enough
X3C09F1-20S	Copies 1% as reference	Passive	None	Provides the comparison signal
TX antenna	Radiates the tone	Passive	None	Illuminates the tag
Tag	Absorbs at its resonances	Passive	None	**Holds the pattern**
RX antenna	Collects the echo	Passive	None	Brings the notches into the circuit
HMC8410	Amplifies weak echo	Whenever powered	None	Lifts notches into the AD8302's range

PART	ITS JOB	ACTIVE WHEN	STM32 CONTROL	ROLE IN THE PATTERN
LFCN + HFCN (RX)	Rejects out-of-band signals	Passive	None	Removes signals from other bands
PE4312	Sets the reference level	Holds its setting	SPI2, LE PE7	Keeps the AD8302 in range; doesn't touch the echo
AD8302	Compares the two signals	Whenever powered	None	**Turns the notches into voltages**
ADC1 + ADC2	Digitise the voltages	Each sample	PA1, PB1	Turns the voltages into numbers

Expected signal levels

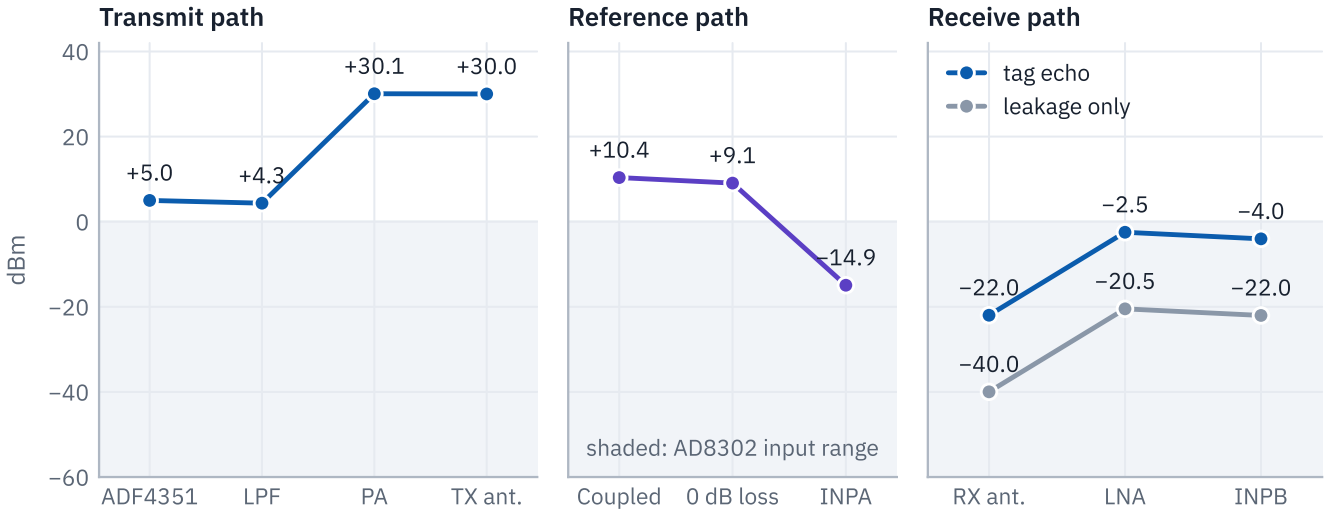


Figure 3. Signal level through the three paths. The receive path shows the echo at 20 cm away from any notch, and the level with leakage alone. Both AD8302 inputs must stay in the shaded range.

3. One read, event by event

This is the recorded sequence of a single `READ_TAG` command, starting with the bytes arriving from the Mac.

RECORDED EVENT	WHAT PHYSICALLY HAPPENS
`A5 04 00 30 00 9A 18`	The request arrives over USB. An interrupt stores the bytes, and the main loop checks the frame's CRC and calls `reader_read_tag()`.
`PA 1`	PE8 goes high. The MAX2235 leaves shutdown and its output ramps up.
`DELAY 1000 µs`	Waits for the PA's ramp to finish (about 600 µs with the 0.068 µF capacitor).

`R5 00580005`
`R4 00AC803C`
`R3 000004B3`
`R2 18004E42`
`R1 08008029`
`R0 00448018`
	First point, 860.000 MHz: all six registers are written, R5 first and R0 last. Writing R0 starts the change of frequency. The tone is now on the antenna.

`poll LD pin`	Reads PE10 every 10 µs, for up to 5000 µs, until the PLL reports lock. A failure adds one to the `unlocked` count, and the reader still takes the measurement.
`DELAY 200 µs`	Lets the frequency and the AD8302 outputs settle.
`8 ADC reads`	ADC1 and ADC2 sample VMAG and VPHS together, 8 times, and each is averaged. The result is converted to dB and degrees and stored.
`R1 080087D1` `R0 004484B8`	**Next point, 860.025 MHz:** only the registers that changed are written. The lock wait, settling and sampling repeat.
`… 199 more points`	Each normally writes R1 and R0 only. At 862.500 MHz, the only integer point, R3 and R2 also change to integer-N settings. They change back at the next point.
`PA 0`	PE8 goes low. The PA shuts down, so the board stops radiating about 1 W.
`R5 … R0`	All six registers are rewritten to return to the resting frequency of 862.5 MHz. The ADF4351 stays on, but only about −44 dBm gets past the PA (*estimate from the 48 dB off-isolation*).
`decode, lookup`	No hardware is involved. The reader works out the 10 bits from the stored results, then looks up the ID in the database (section 6).
`A5 59 00 B0 00 …`	The 92-byte tag report goes back over USB (section 7).

In total, one read writes R0 and R1 202 times each, R2 and R3 4 times each, R4 and R5 twice each. The last write of each comes from returning to the resting frequency.

The two shared SPI wires

The ADF4351 and PE4312 share one clock and one data wire, but each has its own latch line. Both chips shift in every bit that goes past. A chip only acts on what it holds when its own latch line rises: PB12 for the ADF4351, PE7 for the PE4312. So bits meant for one chip pass harmlessly through the other.

Time for one point on real hardware



Figure 4. Estimate, calculated from the code's settings: SPI at 2.625 MHz, ADC at 21 MHz with 96 cycles per sample, and about 80 µs of band selection from the design notes. The real lock time isn't known until the hardware is built. The 100 ms measured on the chip so far is for the simulated build, which also spends time calculating the radio model.

4. Following a 1 and a 0 through the circuit

The table follows two frequencies from the start of the chain to the final decision. f1 has a resonator and f2 does not. The first f1 column is the nominal bit frequency. The second is 25 kHz lower, which is where the signal actually reaches its lowest point. The transmit side is identical for all of them: **the only place the columns start to differ is at the tag.**

STAGE	F1 NOMINAL 860.250	F1 DEEPEST 860.225	F2 NOMINAL 860.750	WHAT IS HAPPENING
Words to ADF4351	`R1 080080C9` `R0 00448080`	`R1 080087D1` `R0 004484F8`	`R1 080080C9` `R0 00448090`	Only the frequency changes. Nothing here depends on the pattern.
ADF4351 output	+5.00 dBm			Same level at every frequency.
After TX low-pass	+4.35 dBm			0.65 dB lost; harmonics reduced.
After PA	+30.07 dBm			Amplifier close to saturation.
At TX antenna	+30.02 dBm			Only 0.05 dB is lost in the coupler.
Reference at INPA	−14.93 dBm			Coupler, then 1.3 dB loss, then 24 dB attenuation.
Tag effect on its own	−20.3 dB	−16.6 dB	−1.6 dB	**The pattern enters here.** f1 has a resonator; f2 has none, so only a neighbour's skirt affects it.
Echo + leakage (channel)	−67.3 dB	−69.4 dB	−53.1 dB	Leakage at −70 dB is added as a vector and fills in the bottom of the notch.
At RX antenna	−37.2 dBm	−39.4 dBm	−23.1 dBm	TX power plus channel.
After LNA	−17.7 dBm	−19.9 dBm	−3.6 dBm	19.5 dB gain.
At INPB	−19.3 dBm	−21.4 dBm	−5.2 dBm	1.55 dB lost in the two filters.
INPA − INPB	+4.3 dB	+6.5 dB	−9.7 dB	Measured back from the voltage below.
VMAG	1025 mV	1086 mV	621 mV	Goes *up* in a notch, because the echo has become weaker.
ADC1 counts	1399	1483	847	Average of 8 samples at the reader.
|phase|	67.2°	32.1°	36.2°	VPHS converted to degrees.
ADC2 counts	1543	2027	1970	Sampled at the same instant as ADC1.
Above background	2.7 dB	0.6 dB	16.9 dB	Before the 3-point median.
Decision	depth 12.3 dB, swing 83.0° **bit = 1**		depth 0.0 dB, swing 15.9° **bit = 0**	Rule: depth at least 6 dB, and swing at least 15° unless depth is 12 dB or more.

Channel and tag-effect rows are calculated from the simulator's formula and agree with the recorded RX levels within 0.02 dB. The VMAG, ADC and phase rows are recorded and include noise.

> **Why the lowest point isn't exactly at f1.** The tag's echo alone is deepest at its resonance, about −72 dB there. That is close to the −70 dB leakage, so the two nearly balance. The final level depends on how their phases add, and in this case they cancel best 25 kHz below f1. The firmware's ±100 kHz search window allows for shifts like this.

The whole sweep

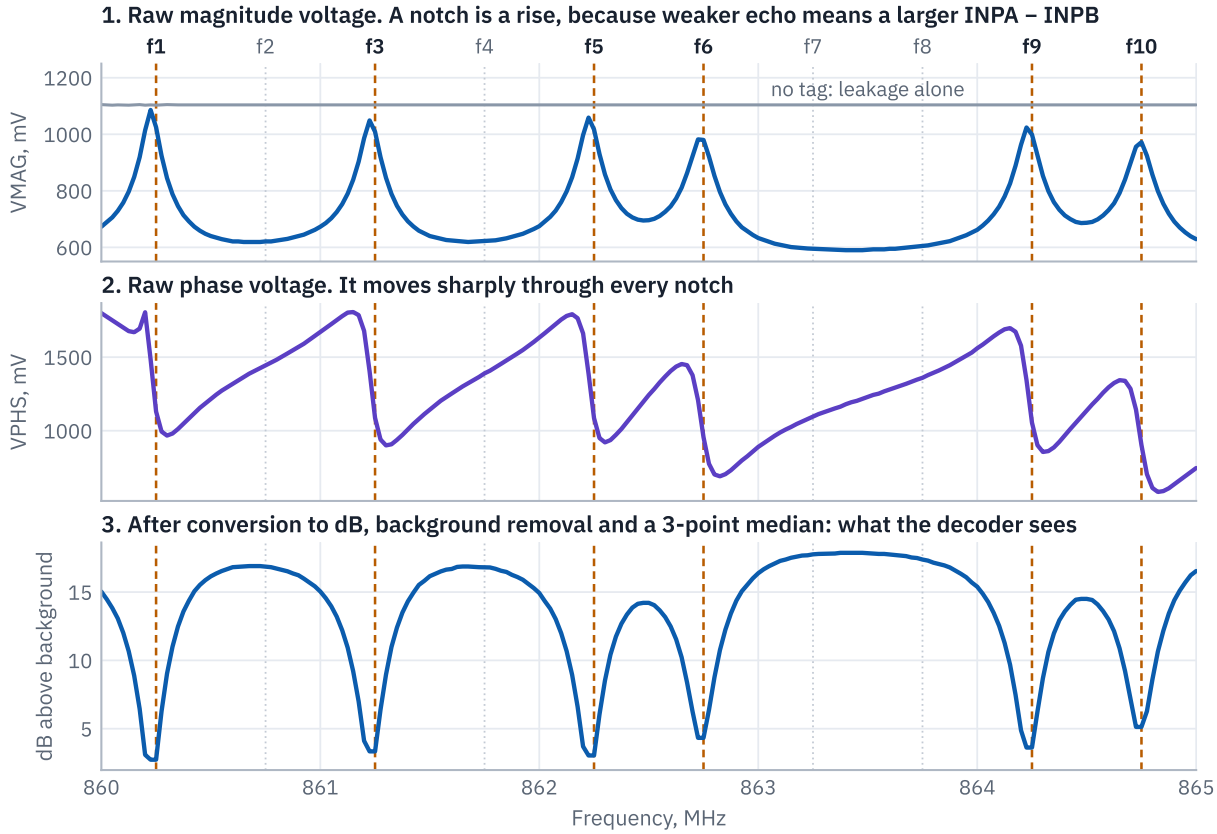


Figure 5. Recorded read of TEN-A at 20 cm. Amber dashed lines are the 1 bits; grey dotted lines are the 0 bits.

BIT	MHZ	DEPTH (DB)	SWING (°)	≥ 6 DB?	BIT	ADDS TO ID
f1	**860.25**	**12.3**	**83.0**	**Yes**	**1**	**1**
f2	860.75	0.0	15.9	No	0	·
f3	**861.25**	**11.7**	**89.8**	**Yes**	**1**	**4**
f4	861.75	0.0	17.4	No	0	·
f5	**862.25**	**11.2**	**86.2**	**Yes**	**1**	**16**
f6	**862.75**	**9.9**	**75.6**	**Yes**	**1**	**32**
f7	863.25	0.0	12.5	No	0	·
f8	863.75	0.0	12.0	No	0	·
f9	**864.25**	**10.9**	**83.4**	**Yes**	**1**	**256**
f10	**864.75**	**9.4**	**75.2**	**Yes**	**1**	**512**

f2 and f4 have phase swings just above 15° and are still 0. They are on the skirt of a neighbouring notch, but the depth test fails. A bit can't be 1 on phase alone.

5. Every part, in signal order

Each part below has the same layout: its job in this circuit, what is inside it, what it does during a read, and how a fault would show up. The inside descriptions are simplified. Check the manufacturer's datasheet before relying on a detail.

5.1 Mac app, SDK and USB link

Job	Lets you control the reader and see results
Connection	Micro-USB on CN5, seen by the Mac as a serial port. Mini-USB on CN1 is only for flashing and debugging.
Code	`ChiplessKit` on the Mac; `protocol.c`, `commands.c`, `app.c` on the STM32

Everything travels as frames: sync byte `A5`, length, command, status, payload and a CRC-16 checksum. The checksum lets the reader reject damaged data and find the next valid frame. Replies use the command number plus `0x80`. In continuous mode, the reader sends a `0xC0` event by itself each time it finds a tag.

USB bytes arrive through an interrupt that fills a ring buffer. The main loop reads that buffer, so commands are handled one at a time and a new one waits until the current sweep has finished. The database lives on the STM32, so identifying a tag doesn't depend on the Mac.

If it fails: the app can't find the reader. Check you are on CN5, not CN1. Corrupted frames raise `frames_err` and get a `CRC_ERROR` reply.

5.2 STM32F407

Job	The brain: controls the RF parts, reads the voltages, decodes the bits, stores the database
Clocks	8 MHz → PLL → 168 MHz CPU, 48 MHz USB, 42 MHz APB1 (SPI2), 84 MHz APB2 (ADC)
Peripherals	SPI2 at 2.625 MHz; ADC1 and ADC2 at 21 MHz; USB CDC; flash sector 1 (16 KB at `0x08004000`)

SIGNAL	PIN	GOES TO	MEANING
SPI2 SCK	PB13	ADF4351 CLK, PE4312 Clock	Shared clock
SPI2 MOSI	PB15	ADF4351 DATA, PE4312 Data	Shared data, MSB first
ADF LE	PB12	ADF4351 LE	Rising edge loads a register
ADF CE	PE9	ADF4351 CE	High = chip enabled
ADF LD	PE10 (input)	ADF4351 LD	High = PLL locked. A pull-down makes "not connected" read as unlocked
PE4312 LE	PE7	PE4312 LE	Rising edge applies the attenuation
PA enable	PE8	MAX2235 SHDN	High = PA on
VMAG	PA1	ADC1_IN1	Level difference voltage
VPHS	PB1	ADC2_IN9	Phase difference voltage

Start-up order matters. The recorded boot sequence is: PA off, attenuator to 24 dB, then ADF4351 programmed. The AD8302 is protected before anything can transmit. The saved settings and database are loaded from flash just before this sequence, so it uses your saved values.

If it fails: the green LED doesn't light and USB doesn't appear. The firmware itself has been tested on the real board, so with RF parts fitted, suspect wiring and shared power first.

5.3 ADF4351 synthesizer and its 25 MHz reference

Job Generates the tone at any frequency the STM32 asks for
In 25 MHz reference (check your module), SPI words
Out +5 dBm tone by default; −4, −1, +2 or +5 dBm available. Lock signal on LD

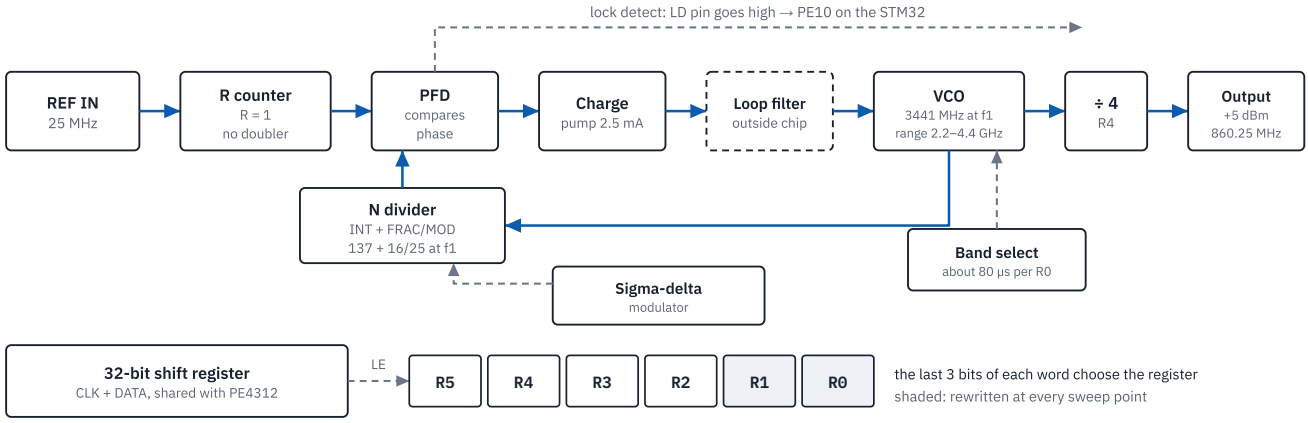


Figure 6. The phase-locked loop inside the ADF4351, with the values at f1.

A VCO is an oscillator whose frequency follows a control voltage. On its own it drifts, so the chip divides its output by N and compares the result with the 25 MHz reference. Any error pushes charge into the loop filter, which corrects the control voltage until the two match. The VCO is then held at exactly N × 25 MHz.

N can be a fraction, because the divider switches rapidly between whole numbers under the control of a sigma-delta modulator so that the average is exact. The VCO only runs from 2.2 to 4.4 GHz, so the firmware uses it at four times the wanted frequency and divides by 4.

For f1: 860.25 MHz × 4 = 3441 MHz. 3441 ÷ 25 = 137.64 = 137 + 16/25. Those numbers are packed into the registers:

REG	WORD AT F1	CONTENTS	REWRITTEN
R0	`00448080`	INT = 137, FRAC = 16. Writing R0 starts the change, including VCO band selection	Every point
R1	`080080C9`	Prescaler 8/9, phase word 1, MOD = 25	Every point in this sweep
R2	`18004E42`	MUXOUT lock detect, R = 1, charge pump 2.5 mA, lock-detect settings, positive phase detector polarity	Fractional/integer change
R3	`000004B3`	Clock divider 150, anti-backlash setting	Fractional/integer change
R4	`00AC803C`	Feedback from VCO, divide by 4, band-select divider 200 (125 kHz), RF output on, +5 dBm	First point only
R5	`00580005`	LD pin = digital lock detect, reserved bits	First point only

BIT	MHZ	VCO MHZ	INT	FRAC/MOD	R0	R1
f1	860.25	3441	137	16/25	`00448080`	`080080C9`
f2	860.75	3443	137	18/25	`00448090`	`080080C9`
f3	861.25	3445	137	4/5	`00448020`	`08008029`
f4	861.75	3447	137	22/25	`004480B0`	`080080C9`
f5	862.25	3449	137	24/25	`004480C0`	`080080C9`
f6	862.75	3451	138	1/25	`00450008`	`080080C9`
f7	863.25	3453	138	3/25	`00450018`	`080080C9`
f8	863.75	3455	138	1/5	`00450008`	`08008029`
f9	864.25	3457	138	7/25	`00450038`	`080080C9`
f10	864.75	3459	138	9/25	`00450048`	`080080C9`

Recorded from the firmware's `SET_LO` replies. The firmware calculates these at run time; there is no lookup table.

Reference accuracy. A 20–50 ppm crystal error moves the band by 17–43 kHz, which is well inside the ±100 kHz search window.

If it fails: the `unlocked` count in the tag report is above 0. If it is every point, check the SPI wiring, CE, the reference and the LD wire. If it is a few points, raise the settle time, and check the loop filter. A wrong crystal value moves every frequency by the same ratio.

5.4 LFCN-1000+ low-pass filter (transmit)

Job	Removes harmonics of the tone before the PA
Level	+5.00 dBm in, +4.35 dBm out (0.65 dB loss, interpolated from the datasheet)

This is a passive filter made of inductors and capacitors, with no power or control. The ADF4351's divided output contains strong harmonics at multiples of the tone, such as 1.72 and 2.58 GHz. The filter passes 860–865 MHz and rejects those, so the PA and antenna don't amplify and radiate them.

If it fails: the measurement could still look normal, since the AD8302 mainly follows the strongest signal. A wrong or damaged filter shows up as low output power, or as harmonics on a spectrum analyser.

5.5 MAX2235 power amplifier

Job	Raises the tone to about 1 W, so the very weak echo can be measured
Supply	3.6 V for the curve used in the firmware
Control	SHDN from PE8. GC is unconnected, which leaves gain at maximum
Level	+4.35 dBm in, +30.07 dBm out

A silicon multi-stage RF amplifier. It is **compressed** here: near its maximum output, more input gives little more output. That's why 9 dB of change at the ADF4351 moves the antenna power by only about 3 dB.

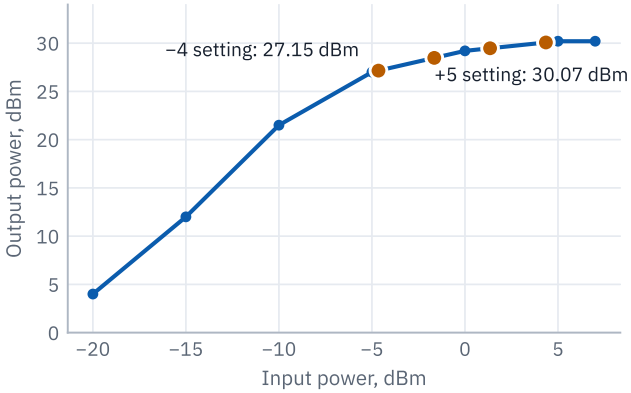


Figure 7. Curve from the firmware's datasheet points. Amber marks the four ADF4351 settings.

ADF4351 SETTING	PA INPUT (DBM)	PA OUTPUT (DBM)	TX ANTENNA (DBM)	PA GAIN	LOWEST ALLOWED ATTENUATOR
−4 dBm	−4.65	+27.15	+27.10	31.8 dB	6.5 dB
−1 dBm	−1.65	+28.47	+28.42	30.1 dB	7.5 dB
+2 dBm	+1.35	+29.47	+29.42	28.1 dB	8.5 dB
+5 dBm	+4.35	+30.07	+30.02	25.7 dB	9.5 dB

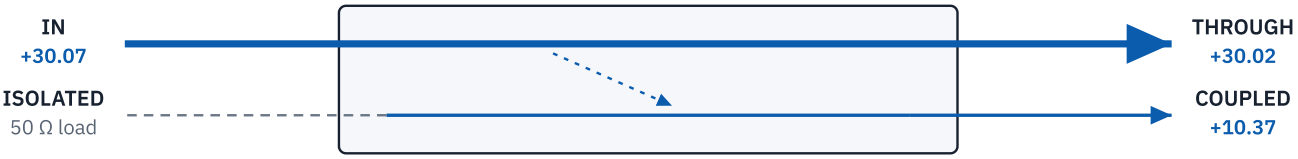
Autoramp and shutdown. The chip ramps its power up and down over about 600 µs to avoid a burst of splatter across nearby channels, which is why the firmware waits 1 ms. In shutdown its output is about 48 dB lower. The firmware uses this to keep the PA off except during a measurement.

Power. With the 47% efficiency Analog Devices lists, 1 W of RF needs about 2.1 W, or about 0.6 A at 3.6 V (*estimate*). That is too much for the Discovery board's own supply, so give the PA its own supply and a shared ground.

If it fails: the tone is weak, or the echo level doesn't change when the PA is switched. Too low a level is also possible if the output match is wrong: the datasheet circuit is matched at 836 MHz, so check the output inductor for 860–865 MHz.

5.6 X3C09F1-20S directional coupler

Job	Copies a small, known part of the transmitted tone for use as the reference
Level	+30.07 dBm in; +30.02 dBm through; +10.37 dBm coupled
Rating	20 dB coupling, 700 MHz–1 GHz, 25 W



1% of the power is copied to the lower line (20 dB); 99% continues to the antenna

Figure 8. The four ports. A directional coupler responds mainly to the wave travelling in one direction.

Two transmission lines run close together inside the part, so a little of the wave in the main line transfers to the second line. It's directional: the forward wave mostly appears at the coupled port, and a wave reflected from the antenna mostly goes to the isolated port, which must end in a 50 Ω load.

This is what makes the measurement work. The AD8302 needs to know what was actually transmitted, including any PA drift. Because the reference is copied *after* the PA, any change in the transmitted tone affects both inputs equally and cancels out in the comparison.

If it fails: a reversed part gives a very low reference level. A poor antenna match also leaks into the reference. As an *estimate*, with 10 dB antenna return loss and 20 dB directivity, that's about ±0.3 dB and ±2° (distributors list 20 or 25 dB directivity).

5.7 TX antenna

Job	Turns the electrical tone into a radio wave aimed at the tag
Level	About +30 dBm (1 W), only during measurements
Wavelength	About 34.8 cm at 862.5 MHz

The repository doesn't specify the antennas, so these are general points. The antenna must cover 860–865 MHz and be well matched at 50 Ω. Its placement relative to the RX antenna matters as much as the antenna itself, because every bit of direct coupling becomes leakage. Separation, shielding and crossed polarisation are the common ways to reduce it.

> **Licensing.** In India, the licence-exempt RFID band starts at 865 MHz, so most of 860–865 MHz needs authorisation to radiate, and 1 W is a substantial signal. Until that's settled, test in a shielded enclosure or with cabled connections, as the design notes advise.

5.8 The chipless tag

Job	Holds the pattern, as copper
Power	None. It only reacts to the reader's own tone
Model	−52 dB echo at 20 cm, −20 dB notch, Q = 2000: all estimates

A resonator is a copper shape that stores energy at one frequency, as a tuning fork does for one note. When the reader's tone reaches that frequency, the resonator takes energy out and dissipates it, so less comes back. At other frequencies it has little effect. The simulator describes each one as:

$$H(f) = 1 - \kappa \,/\, (1 + j \cdot 2Q \cdot (f - f_k) \,/\, f_k)$$

With κ = 0.9, the echo falls to 0.1 of its level at resonance, which is −20 dB. All ten effects multiply, so one resonator's skirt affects its neighbours.

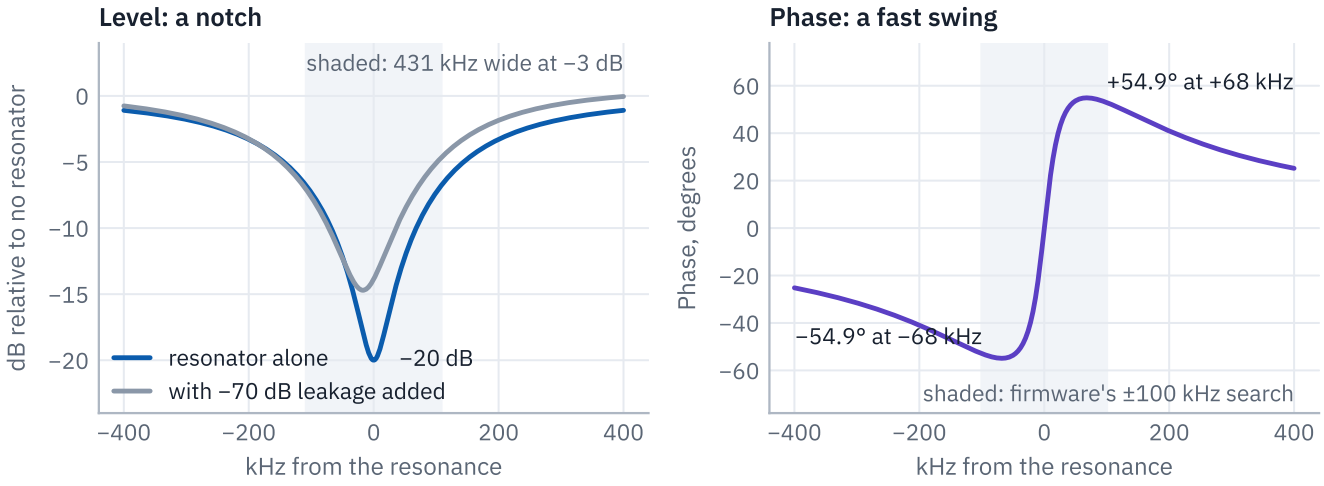


Figure 9. One resonator with Q = 2000, calculated from the simulator's formula. With leakage added, this lone notch is about 15 dB deep at 20 cm instead of 20 dB. Neighbouring notches on a full tag reduce it further, to the recorded 9–12 dB. Phase is why the reader has a second check: only a real resonance moves both.

Q decides whether 10 bits are possible. Notch width is about f/Q. With Q = 2000 it is 431 kHz and fits the 500 kHz spacing. With Q = 250 it is 3.45 MHz, and all ten merge together. Your design notes found 1250 is needed to read at all and about 2000 to read with margin, while printed copper is usually in the low hundreds. **This is the biggest open question for the design.**

If it fails: `0000000000` can't be used, as it looks exactly like no tag. Two tags at once mix their notches. A manufacturing shift of 0.5% is more than 4 MHz, nearly the whole band, so measure real tags and enter their real frequencies.

5.9 RX antenna

Job	Collects the echo. It also unavoidably collects leakage from the TX antenna
Level	−22 dBm echo at 20 cm; −40 dBm leakage (both estimates)

Two signals arrive together as one: the echo carrying the notches, and leakage that doesn't. They add as vectors, so depending on phase they reinforce or cancel. That is why the leakage limits how deep a notch can appear, and why the recorded notches are 9–12 dB and not 20 dB.

The isolation is critical. Your design notes calculate that at least **48 dB** is needed to keep INPB under 0 dBm, and at least **38 dB** to stay under the AD8302's +10 dBm absolute maximum. Measure it before connecting INPB.

5.10 HMC8410 low-noise amplifier

Job	Amplifies the echo while adding as little noise as possible
Figures	19.5 dB gain, 1.1 dB noise figure, 0.01–10 GHz
Supply	5 V at 65 mA, with VGG1 adjusted to set that current
Level	−22 dBm in, −2.5 dBm out (echo at 20 cm)

A GaAs pHEMT transistor stage. It comes first in the receive chain because the first stage's noise matters most. As a rough *estimate*, with the noise figure and about 300 MHz of filter bandwidth, the noise at INPB is around −70 dBm. That's below the AD8302's −60 dBm floor, so the LNA's noise isn't the limit. Leakage is.

No firmware control. The LNA runs whenever powered, and the firmware accounts for its gain only in the RSSI figure. Notch depth doesn't depend on that value.

> **Bias order.** This type of transistor can draw too much current if its drain is powered without its gate bias. Analog Devices' procedure for the sibling ADH8410S sets the gate negative first, then applies VDD, raises VGG1 to reach 65 mA, and only then applies RF. Confirm this in the HMC8410's own datasheet or your module's notes. Absolute maximum RF input is +20 dBm.

If it fails: received power is about 20 dB lower than expected, or the current isn't 65 mA.

5.11 Receive filters: LFCN-1000+ and HFCN-650+

Job	A low-pass and a high-pass that together act as a wide band-pass
Level	−2.5 dBm in, −4.0 dBm out at INPB (1.55 dB loss in total)

The AD8302 responds to all the power it receives, whatever its frequency, so an out-of-band signal could disturb the reading. These filters reject signals well below and above the band. The block is wide compared with the 5 MHz band, so it can't reject a strong signal close by, and it doesn't help with leakage, which is at the same frequency as the tone.

If it fails: an unexpectedly high or changing INPB level, with the same reading whether the TX is on or off, points to an outside signal.

5.12 PE4312 digital attenuator (reference path)

Job	Sets the reference to a safe level that suits the AD8302
Range	0–31.5 dB in 0.5 dB steps, plus 1.3 dB of insertion loss
Control	SPI2 + LE on PE7, with P/S high for serial mode
Level	+10.37 dBm in, −14.93 dBm out at 24 dB

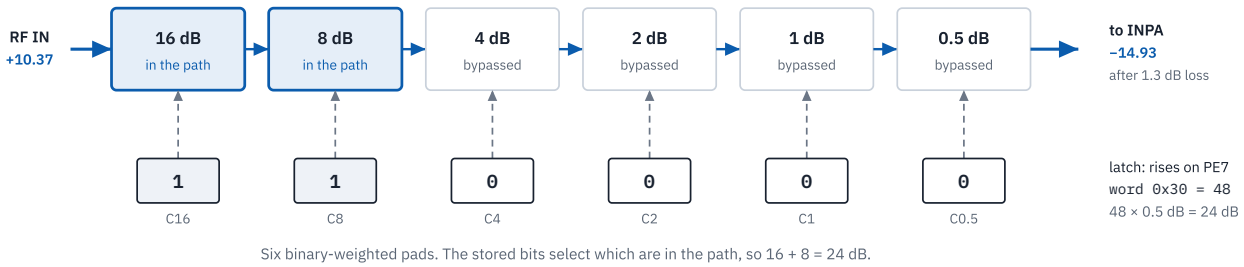


Figure 10. The default setting, 24 dB. The STM32 clocks 8 bits. The first two are padding and fall out of the 6-bit shift register.

It affects only the reference. It doesn't change how strong the echo is. It moves the point the echo is compared against, so 1 dB more attenuation lowers INPA − INPB by 1 dB.

Why the firmware protects it. At 0 dB, INPA would be about +9 dBm, close to the AD8302's +10 dBm absolute maximum. The firmware refuses anything below the calculated minimum, which is 9.5 dB at the default power. The recording confirms it: `SET_RX_CONFIG` rejects 9.0 dB with `OUT_OF_RANGE` and accepts 9.5 dB, giving −0.4 dBm.

Auto-level sweeps, finds the middle of the measured range, and picks the setting that centres it. **Attenuator scan** steps through settings and records VMAG. A straight line proves the wiring, and its slope gives your AD8302's real mV/dB.

> Firmware can't protect the AD8302 before it has started. `board.h` says to strap the C0.5–C16 inputs for high attenuation so that the power-up state is safe.

If it fails: a flat attenuator scan means no control: check PE7, P/S and the data wire. Steps that look wrong may mean a bit-order or wiring fault.

5.13 AD8302 gain and phase detector

Job	The heart of the receiver: compares the reference with the echo, and gives two DC voltages
In	INPA (reference) and INPB (echo), each −60 to 0 dBm, +10 dBm absolute maximum
Out	VMAG 28.7 mV/dB, VPHS 10.1 mV/° (at 900 MHz), both centred on 900 mV
Set-up	VMAG tied to MSET, VPHS tied to PSET; inputs AC-coupled with 52.3 Ω shunts

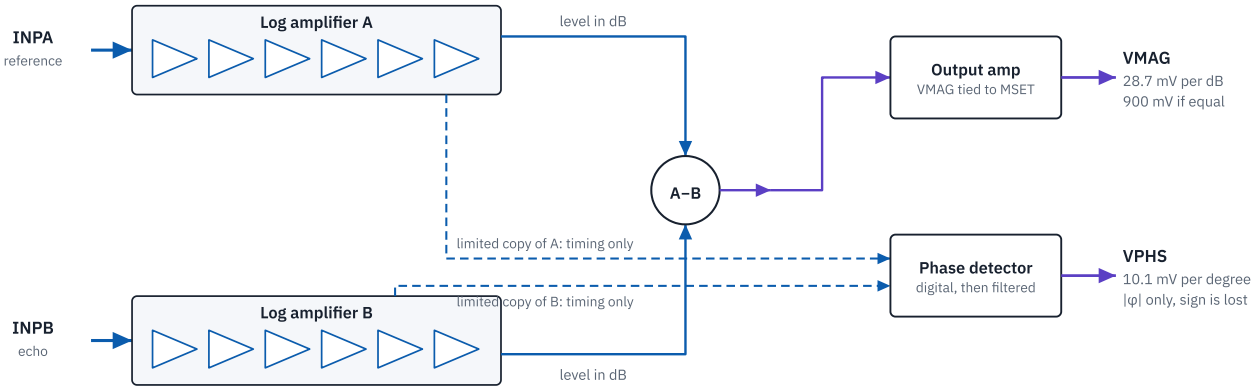


Figure 11. Simplified inside of the AD8302.

Magnitude. A log amplifier gives an output proportional to the input level in dB. There are two matched ones. Subtracting two dB values gives their ratio directly, so VMAG depends on the *difference* between the inputs. This has an important effect: **when the echo drops in a notch, VMAG goes up.**

Phase. The last stages of the log amplifiers act as limiters, which remove the amplitude and keep only the timing. These two square-wave signals feed a digital phase detector, and its filtered output is VPHS.

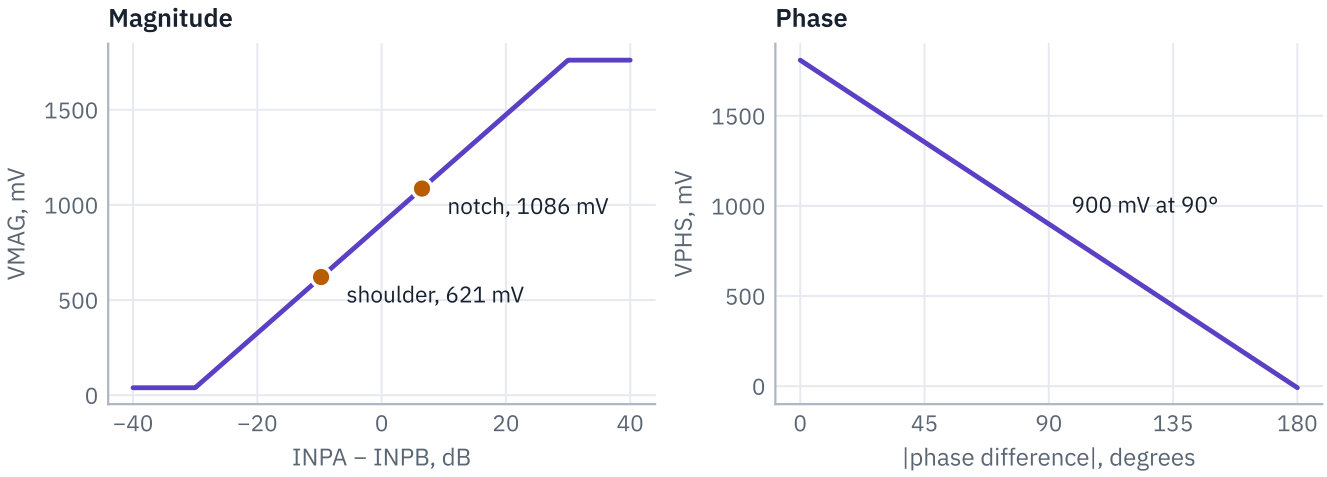


Figure 12. The two transfer curves in the firmware's calibration values. Amber marks the recorded shoulder and notch at the first two bits.

Two limits shape the whole decoding design:

- **Only the size of the phase is available.** The chip can't tell +90° from −90°, and the output folds at 0° and 180°. So the firmware uses phase *swing* across a bit, which survives the fold, and not absolute phase.
- **The range is ±30 dB around the reference.** With INPA at −14.93 dBm, INPB must be between −44.9 and 0 dBm (0 dBm is the specified limit). This window is what the attenuator moves.

If it fails: VMAG stuck near 0 or 1.8 V means an input is far out of range: `clipped` rises. VMAG that doesn't react to the attenuator means no reference.

5.14 ADC1 + ADC2

Job	Converts VMAG and VPHS to numbers at the same instant
Mode	Dual regular simultaneous, 12-bit, 21 MHz clock, 84-cycle sampling, 8 samples averaged
Scale	About 3.0 V full scale, measured at start-up using the internal reference

Sampling together matters because VMAG and VPHS then describe one moment. One count is 0.733 mV, which is 0.026 dB or 0.073°. The AD8302's outputs stop at 1.8 V, so about 2457 of the 4095 counts are used. That's far finer than the noise, so the ADC isn't a limit.

VDDA is measured at start-up rather than assumed, so dB values stay correct if the board's supply is 2.9 or 3.1 V. Counts become millivolts with `counts × VDDA ÷ 4095`.

If it fails: if the two inputs are swapped, magnitude and phase will be exchanged and no notches will appear in the right place. Check the raw values on the Calibration screen.

5.15 Power supplies

PART	SUPPLY	CURRENT	SOURCE OF THE FIGURE
MAX2235	3.6 V	About 0.6 A when on, under 1 µA in shutdown	Estimate from 47% efficiency; shutdown from Analog Devices
HMC8410	5 V + negative gate bias	65 mA	`board.h`, datasheet
AD8302	2.7–5.5 V	Check its datasheet	`board.h`
ADF4351, PE4312	As per your modules	Check your modules	Not recorded in the repository
STM32 board	USB	·	Discovery board

All supplies need a common ground. The AD8302's outputs reach only 1.8 V, so they are safe for the ADC at any permitted AD8302 supply.

6. Turning the voltages into the pattern

Once the sweep ends, the STM32 holds 201 pairs of numbers and no more hardware is involved. This is the point where notches become bits.

Step 1. Counts to voltage to dB and degrees
Using the recorded point at f2 (860.750 MHz): 847 counts × 3000 ÷ 4095 = 620.5 mV. Then (620.5 − 900) ÷ 28.7 = −9.74 dB, the INPA − INPB difference.

Step 2. Work back to the RX antenna
INPB = −14.93 − (−9.74) = −5.19 dBm. Removing the LNA gain and filter loss gives −5.19 − 19.5 + 1.55 = **−23.14 dBm**, the recorded value. This number depends on the calibration. The notch depth, measured within one sweep, mostly doesn't.

Step 3. Remove the background
If a no-tag sweep was captured, it is subtracted in dB at each point. This flattens the frequency shape of the antennas and room.

> **It doesn't remove the leakage signal itself.** Subtracting dB values at each frequency corrects the baseline, but the leakage has already added to the echo as a vector inside the RX antenna. That's why notches still measure 9–12 dB, not 20. In the recording, the read with no background gave nearly identical depths (f1: 12.3 dB both ways), because the simulated background is flat. It may matter more with real antennas, which aren't flat.

Step 4. A 3-point median
Each point becomes the middle value of itself and its two neighbours, which removes single-point noise spikes. It also trims a very narrow bottom: at f1, the lowest raw value 0.6 dB becomes 2.7 dB.

Step 5. Measure each bit

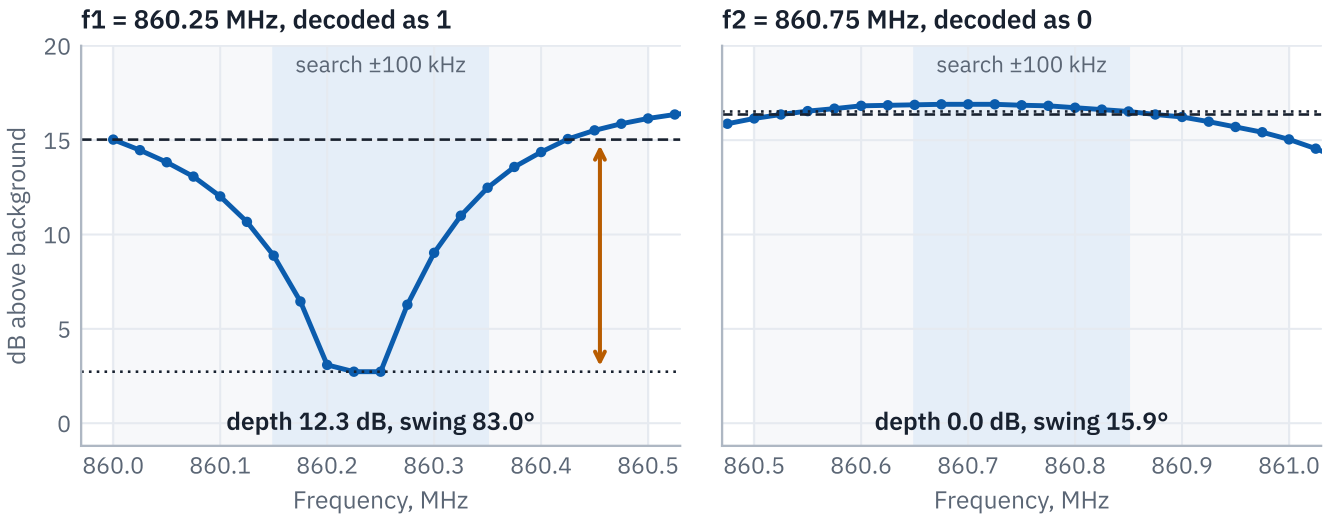


Figure 13. Recorded data. The lowest level is found inside the blue search window. The highest level is then found on each side, out to the midpoint with the next bit. Depth is the *lower* shoulder minus the lowest level.

Taking the lower shoulder is the key detail. f2 sits on the gently sloping stretch between the notches at f1 and f3, so its level changes a little across its window. Using the lower shoulder stops that slope from counting as a dip, and its depth correctly stays near zero.

Step 6. Decide

```
phase_ok = (phase_thr == 0) or (swing >= 15°) or (depth >= 12 dB)
bit = 1  if  depth >= 6 dB  and  phase_ok
```

All three limits are settings under Reader Setup. Margin is the smallest distance of any bit from the 6 dB threshold. It's 3.4 dB here, from f10 at 9.4 dB, and it's a good measure of how trustworthy the read is.

Step 7. Look it up

`tagdb_match()` XORs the decoded ID with each stored ID of the same length and counts differing bits. By default no differences are allowed. `0x335` matches entry 2, `TEN-A`. A pattern not in the database is reported as not matched, and never given a wrong name.

All the built-in tags

TAG	PATTERN	DECODED	RSSI	MARGIN	MATCHED
SINGLE-1	`1`	`0x001`	−21.9 dBm	8.5 dB	Yes
DOUBLE-10	`10`	`0x001`	−21.8 dBm	5.9 dB	Yes
TEN-A	`1010110011`	`0x335`	−25.2 dBm	3.4 dB	Yes
TEN-B	`0101001100`	`0x0CA`	−23.8 dBm	3.6 dB	Yes
TEN-C	`1111111111`	`0x3FF`	−28.8 dBm	2.8 dB	Yes
TEN-D	`1010101010`	`0x155`	−24.5 dBm	5.1 dB	Yes
TEN-E	`1000000001`	`0x201`	−22.3 dBm	5.6 dB	Yes

Recorded at 20 cm, Q 2000, 0.05 dB noise, one read each.

7. The pattern as bytes on USB

These are the only places the pattern is sent as a message. All bytes are recorded. Multi-byte values are little-endian, so the lowest byte comes first.

Mac to STM32: store the pattern

```
A5 0E 00 26 00 35 03 00 00 0A 54 45 4E 2D 41 D4 45
```

BYTES	FIELD	MEANING
`A5`	Sync	Start of frame
`0E 00`	Length	14 bytes from the command to the CRC
`26`	Command	`DB_PUT`
`00`	Status	Always 0 in a request
`35 03 00 00`	ID	`0x335`**, which is the pattern 1010110011**
`0A`	Bits	10
`54 45 4E 2D 41`	Name	"TEN-A" in ASCII
`D4 45`	CRC	CRC-16/CCITT-FALSE over command through name

The reply is `A5 08 00 A6 00 02 00 07 00 26 16`: index 2, count 7. So far this is only in RAM. To keep it after power-off, the app sends `NV_SAVE`, `A5 04 00 06 00 A9 B7`. The STM32 erases flash sector 1 and writes the settings, calibration and database, then verifies them. The CPU pauses during the erase. The reply is `A5 04 00 86 00 31 AC`.

Mac to STM32: ask for a read

```
A5 04 00 30 00 9A 18
```

Sync, length 4, command `READ_TAG` (0x30), status 0, no payload, CRC. **This contains no pattern.** It only says "read now".

STM32 to Mac: the result

```
A5 59 00 B0 00 01 00 00 00 01 0A 35 03 00 00 04 FF 41 01 22 00 29 00 C9 00 01 00 00 00 00 7B 00
00 00 75 00 00 00 70 00 63 00 00 00 00 00 6D 00 5E 00 3E 03 9F 00 82 03 AE 00 5E 03 F4 02 7D 00
78 00 42 03 F0 02 01 00 02 00 54 45 4E 2D 41 00 00 00 00 00 00 00 00 00 00 00 49 A0
```

BYTES	FIELD	VALUE
`A5 59 00 B0 00`	Header	Sync, length 89, reply to `READ_TAG`, status OK
`01 00 00 00`	seq	Read number 1
`01` · `0A`	found · bits	Tag found, 10 bits
`35 03 00 00`	id	`0x335`**: the pattern, recovered from the echo**
`04 FF`	rssi	−252, so −25.2 dBm, averaged across the band
`41 01`	phase	321, so 32.1° at the deepest notch
`22 00`	margin	34, so 3.4 dB
`29 00`	sweep_ms	41 ms in this recording, which counts only the requested delays. About 100 ms on the chip with the simulated build

BYTES	FIELD	VALUE
`C9 00` · `01`	points · bg	201 points, background used
`00 00` · `00 00`	clipped · unlocked	No clipped points and no unlocked points
`7B 00 00 00 75 00 …`	depth × 10	10 values: 12.3, 0.0, 11.7, 0.0, …
`3E 03 9F 00 82 03 …`	swing × 10	10 values: 83.0, 15.9, 89.8, …
`01` · `00` · `02 00`	match	Matched, 0 bit errors, database entry 2
`54 45 4E 2D 41 00 …`	name	"TEN-A", padded to 16 bytes
`49 A0`	CRC	Checksum

8. Limits, risks and faults

Distance: what the model predicts

The firmware was run at a range of distances, capturing a new background each time, with TEN-A at Q 2000. The table shows a **usable window of about 10–30 cm**, with failures on both sides. This isn't stated in your existing documents, and it depends on the estimated channel values.

DISTANCE	ECHO	INPB (ECHO ONLY)	1-BIT DEPTHS	RESULT	WHY
5 cm	−28 dB	+20 dBm	0.0–0.8 dB	**No read**	Receiver overloaded: the levels pile up at the simulator's +3 dBm ceiling, so the notches disappear.
10 cm	−40 dB	+8 dBm	13.4–14.2 dB	**Correct**	But INPB is above 0 dBm. See the warning below.
15 cm	−47 dB	+1 dBm	11.3–13.9 dB	**Correct**	INPB is still above 0 dBm.
20 cm	−52 dB	−4 dBm	9.4–12.3 dB	**Correct**	The default test point.
30 cm	−59 dB	−11 dBm	8.3–17.5 dB	**Correct**	Weakest correct read.
40 cm	−64 dB	−16 dBm	4.9–10.3 dB	**Wrong pattern**	f6 and f10 fall below 6 dB, so 0x115 is decoded. It is not in the database, so no name is given.
50 cm	−68 dB	−20 dBm	3.2–3.8 dB	**No read**	Echo is now close to the leakage level.
100 cm	−80 dB	−32 dBm	0.0–0.6 dB	**No read**	Echo is well below leakage.
300 cm	−99 dB	−51 dBm	0.0–0.1 dB	**No read**	Only leakage remains.

> **A close tag may overload the AD8302, and the tag report won't warn you.** Your design notes cover leakage, but the same limits apply to the echo. With these estimates, the tag's echo alone takes INPB above the 0 dBm limit at under about **16 cm**, and above the +10 dBm absolute maximum at under about **9 cm**. The simulator read correctly at 10 cm, but real hardware might not, and could be damaged. The `RF_INPB_HIGH` flag is returned by calibration commands such as `MEASURE` and `AUTO_LEVEL`, but not in the tag report. The report contains only `clipped`, which won't catch this condition. So check INPB on the Calibration screen. **Keep the 20 dB pad on INPB until real levels have been measured.**

Other things to know

- **The far limit comes from leakage, not sensitivity.** The echo equals the leakage at about 56 cm. Better antenna isolation would extend the range more than extra gain would.
- **Near field.** At 20 cm the tag is under one wavelength from the antennas, so the $1/r^4$ model is only a rough guide at these distances.
- **The simulated clip flag can't trigger.** The simulator limits the difference to ±30 dB, which with the 28.7 mV/dB slope gives 39–1761 mV. The firmware's clip thresholds are 30 and 1770 mV. So `clipped = 0` in simulated runs tells you nothing. On hardware the chip's real output range should trigger it, but that needs checking.
- **PA enable during reset.** `main.c` drives PE8 low when it starts, but the pin floats before that. Consider a pull-down on the SHDN line, so the PA can't turn on while the STM32 is in reset. Check how SHDN behaves in the datasheet.

Suggested bring-up order

1. **Measure a real tag on a VNA:** frequency, notch depth and Q. This decides whether 10 bits in 5 MHz is possible at all.
2. **Measure antenna isolation** before connecting INPB.

3. **Check control only:** with RF outputs into a 50 Ω load, flash the hardware build, confirm SPI, LE and PLL lock.
4. **Check the reference path:** run the attenuator scan and confirm a straight line.
5. **Add the receive path** with a 20 dB pad on INPB, then remove it only if the levels allow it.
6. **Read a tag** in a shielded enclosure or with cabled connections, starting from the far end of the range and moving closer.

Symptom guide

WHAT YOU SEE	LOOK AT FIRST
`unlocked` equals the number of points	ADF4351: SPI wires, CE, reference, LD wire on PE10
`unlocked` is a few points	Settle time, loop filter, module supply noise
Same flat result with or without a tag	Leakage too high, antennas, or Q too low to form notches
Attenuator scan is flat	PE4312: PE7, P/S pin, data wire, or no signal on the coupled port
Attenuator scan is straight, but slope isn't 28.7 mV/dB	Normal part variation. Enter the measured slope in calibration
`clipped` is high	AD8302 is outside ±30 dB. Run Auto-level, then check levels
Pattern is close but a few bits are wrong	Bit frequencies don't match the tag, or margin is too small. Check depths under Read Tags
All notches appear shifted the same amount	Reference value: check `ADF_REF_KHZ` against your module
Notches are very shallow but RSSI changes with the tag	Leakage close to the echo level, or tag too far away
Good pattern but "not in database"	Add the pattern. Check the number of bits matches the profile
Tone or spurs near 862.5 MHz disturb f5 and f6	Integer-N point. See the note in `DESIGN_NOTES.md`

9. What is real, sources and terms

ITEM	STATUS
USB frames, CRC, commands, database, flash storage	Tested on the real STM32
Register values for the ADF4351	Calculated and unit-tested; never loaded into a real chip
Decode logic	Tested on the real STM32, on simulated readings
SPI to real chips, PLL lock, real AD8302 readings	**Not tested**
Echo level, leakage, notch depth, Q	**Estimates**
Chain gains and losses	Datasheet values, not measured

Sources

This repository: `firmware/core/*`, `firmware/stm32/Inc/board.h`, `firmware/stm32/Src/main.c`, `firmware/stm32/Src/platform_stm32.c`, `docs/DESIGN_NOTES.md`, `docs/PROTOCOL.md`.

Checked online for this document: Analog Devices AN-691 (AD8302 log amps and limiter-driven phase detector), AD8302 specifications, ADF4351 product page, MAX2235 product page, MAX2235 article (autoramp), HMC8410 datasheet, ADH8410S datasheet (bias sequence of a sibling part), PE4312 product page, X3C09F1-20S listing.

Not independently checked: general descriptions of how PLLs, couplers, filters and log amplifiers work come from standard RF practice. The LFCN-1000+ and HFCN-650+ figures are the ones in your design notes. The full manufacturer datasheets weren't re-read for this document, so confirm pin-level details there.

Terms

dB, dBm	dB is a ratio: +3 dB is about double the power. dBm is an absolute power: 0 dBm = 1 mW, +30 dBm = 1 W
Notch	A dip in the echo at one frequency, caused by a resonator
Shoulder	The higher level on either side of a notch
Q	Sharpness of a resonator: frequency divided by notch width
Leakage	Tone travelling directly from the TX antenna to the RX antenna
Isolation	How much weaker that leakage is than the transmitted tone
INPA, INPB	AD8302 inputs: reference and echo
VMAG, VPHS	AD8302 outputs: level difference and phase difference
PLL, VCO, PFD	The ADF4351's frequency-control loop, oscillator and phase comparator
Compression	An amplifier near its limit, where more input gives little more output
Margin	How close the least certain bit was to the decision threshold
Absolute maximum	A level that can damage the part, as distinct from the range where it works properly
VNA	Vector network analyser: a lab instrument that measures a part's response directly